

STACK

Q2) Reverse And Merge Two Stacks

DS	Session	Stack	Question Information	Level 1	Challenge 42
Question description					
Rajinikanth organized a technical round interview in an Animation company for the set of computer science candidates.					
the task is to implement stack operations for two stacks and merge the stacks into one.					
Get two sets of stack elements and reverse them, then merge them into one stack.					
Rajinikanth has given the deadline of only 15 minutes to complete the problem.					
Can you Help the candidates to complete the problem within the specified time limit?					
Function Description					
a) push(): Adds the new item at the beginning of the linked list using the first pointer.					
b) pop(): Removes an item from the beginning using the first pointer.					
c) merge(): Links the first pointer second stack as next of the last pointer of the first list.					
Constraints					
$0 < n, m < N$					
$1 < arr[i] < 1000$					
Input Format:					
First line indicates n & m, where n is the number of elements to be pushed into the stack and m is the number of pop operations that need to be performed					
The next line indicates the n number stack elements					
Output Format:					
First line indicates the top of the element of the stack					

CODE:

```
C main.c
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  typedef struct Node {
5      int data;
6      struct Node* next;
7  } Node;
8
9  typedef struct mystack {
10     Node* top;
11 } mystack;
12
13 void push(int data, mystack* ms) {
14     Node* t = (Node*)malloc(sizeof(Node));
15     t->data = data;
16     t->next = ms->top;
17     ms->top = t;
18 }
19
20 int pop(mystack* ms) {
21     if (!ms->top) return -1;
22     Node* t = ms->top;
23     int v = t->data;
24     ms->top = t->next;
25     free(t);
26     return v;
27 }
28
29 void merge(mystack* ms1, mystack* ms2) {
30     if (!ms1->top) {
31         ms1->top = ms2->top;
32         return;
33     }
34     Node* p = ms1->top;
35     while (p->next) p = p->next;
36     p->next = ms2->top;
37 }
38
39 int main(void) {
40     int n, m, x;
41     if (scanf("%d %d", &n, &m) != 2) return 0;
42
43     mystack in1 = {NULL}, in2 = {NULL};
44     mystack rev1 = {NULL}, rev2 = {NULL};
45
46     for (int i = 0; i < n; i++) {
47         scanf("%d", &x);
48         push(x, &in1);
49     }
50     for (int i = 0; i < m; i++) {
51         scanf("%d", &x);
52         push(x, &in2);
53     }
54
55     while (in1.top) push(pop(&in1), &rev1);
56     while (in2.top) push(pop(&in2), &rev2);
57
58     merge(&rev1, &rev2);
59
60     Node* curr = rev1.top;
61     while (curr) {
62         printf("%d ", curr->data);
63         curr = curr->next;
64     }
65     printf("\n");
66
67     return 0;
68 }
```

OUTPUT:

```
PS C:\Users\CH.SC.U4CSE25255\Projects\test> ./main
5 8
3 8 7 6 5
5 6 7 8 3 11 2 3
3 8 7 6 5 5 6 7 8 3 11 2 3
```